SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS: **Total Marks: 20**

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Code of Conduct:

I Y. Abhay-192411219 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Y. Abhay

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding

Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Y. Abhay
192411219

CO2 ASSESSMENT - 3

Question 1

Responsive Web Design.

Responsive Web Design (RWD) is a web design approach that enables a website to automatically adjust its layout and content according to different screen sizes such as mobile phones, tablets, and desktop computers. It ensures that users have a consistent and user friendly experience across all devices.

Importance:

- Provides a better user experience on all devices.
- Eliminates the need for seperate mobile websites
- Improves accessibility and readability.
- Reduce maintenance cost
- Enhances SEO as search engines prefer mobile - friendly websites.

CSS Techniques for Responsiveness:

- Media Queries: Apply different styles based on screen size.
- Flexible layout: Use ~~Flex~~ Flex box or Grid instead of fixing ~~(poisoning)~~ positioning.
- Responsive Images: Set image width to 100% or use max-width.
- Relative Units: Use %, em, rem and vw instead of fixed pixels.
- Viewport Meta Tag: Ensures proper scaling on mobile devices.

Flexbox and Grid:

- Flexbox: Best for one-dimensional layouts (rows and columns), such as navigation bars.
- CSS Grid: Best for two-dimensional layouts (rows and columns), such as dashboards and complete webpage layouts.

Example Layout:

Desktop

Header

Sidebar | Main content

Footer

Mobile

Header

Sidebar

Content

Footer

Sample CSS (Flexbox + Media Query)

```
.container {
    display: flex;
}

@media (max-width: 768px) {
    .container {
        flex-direction: column;
    }
}
```

Improvements:

- Use responsive images
- Add a collapsible navigation menu
- Optimize page loading speed
- Test on different devices/browsers.

Question 2

Client side validation.

Client side validation is the process of checking user input in the browser before sending data to the server. It helps prevent invalid or incomplete data from being submitted.

Purpose:

- Reduces unnecessary server requests.
- Improves user experience by providing immediate feedback.
- Prevents invalid data entry.
- Saves server processing time.

JavaScript Event Handling.

JavaScript events detect user actions and trigger validation.

Common events include:

- onclick
- Onblur
- On keyup
- On submit

Role of Regular Expression (Regex)

Regular expressions are patterns used to validate input formats.

Example:

- Email : abc @gmail.com.
- Phone Number : 9876543210

Javascript

```
let email pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
let phone pattern = /^[0-9]{10}$/;
```

Example:

When a user enters an invalid email, Javascript immediately display an error message such as "Invalid email Address" without refreshing the page.

Improvings:

- Display real-time validation messages.
- Highlight invalid fields.
- Use strong password validation.
- Perform server-side validation in addition to client side validation.
- Use CAPTCHA to prevent automated submissions.

Sample JavaScript (Regex):

```
let email = document.getElementById("email").value;
let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (pattern.test(email))
        alert("Valid");
else
        alert("Invalid");
```

Question 5

CSS Box Model

The CSS Box Model defines how the size and spacing of HTML elements are calculated.

It consists of four components.

1. Content - Actual text or image
2. Padding - space inside the border
3. Border - surrounding the padding.
4. Margin - Space outside the border.

Margin

Border

Padding

Content

CSS positioning techniques

- Stable
- Relative
- Absolute
- Fixed
- Sticky

Causes of Layout breaking.

- Fixed width elements
- Improper use of absolute positioning.
- Missing media queries
- Large margins on padding

images without responsize sizing.

Using Media Queries:

Media queries apply different CSS rules based on screen size.

Example:

```css
@media (max-width: 768px){
    .container{
    flex-direction: column;
    }
}
```

Best practices:

- Use flexbox or grid layouts
- Use relative units (%, rem, vw)
- Apply box-sizing: border-box
- Make images responsive
- Test layouts on multiple devices.

Question 4

Document Object Model (DOM)

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. It allows JavaScript to access, modify, add, or remove HTML elements dynamically.

Role in Web Development:

The DOM enable developers to:

- Change webpage content
- Modify CSS styles.
- Handle users events
- Create interactive web applications
- Update webpage without reloading

DOM Manipulation using JavaScript:

Common methods include.

- getElementById()
- querySelector()
- innerHTML()
- text content
- createElement()
- append child()

Example:

<!> Java Script

document.getElementById("demo").innerHTML = "Welcome to IP";

Output:

Before:

Hello

After clicking a button:

Welcome to SP

Improvements:

- Minimize unnecessary DOM updates
- Use event delegation for better performance
- Reduce page reloads using AJAX or Fetch API
- Optimize animation with CSS instead of excessive JavaScript.
- Cache frequently accessed DOM elements.

Sample JavaScript:

</> JavaScript

```
document.getElementById("demo").innerHTML = "Welcome";
```

Demonstrates:

- DOM Manipulation
- Dynamic content Update.